MAIS-MVP Bug Severity Guide

P0, P1, and P2 triage rules with project-specific examples for new employees.

Audience	New MAIS-MVP employees, debugging agents, QA reviewers, and release coordinators
Primary source	AGENTS.md coordination contract, README project surface list, and current content QA severity rules
Scope	Product, API, content, release, security, data, and documentation bugs inside MAIS-MVP
Owning role	S10 docs/report lead maintains this guideline; each bug still routes to its feature or platform owner

Use this first: Classify by the worst credible user or release impact, not by how hard the fix looks. When evidence is incomplete, choose the higher severity until the blast radius is proven smaller.

Quick Severity Matrix

Severity	Plain-English meaning	Default response	Good MAIS-MVP examples
P0	Stop-the-line critical incident or release blocker. Production use, student data, secrets, grading correctness, or release safety is unsafe.	Immediate owner notification, release hold or feature disable, smallest safe mitigation, full proof before reopen.	Global login failure; secret exposed; active release build cannot complete; live answer key mismatch at scale; uncontrolled live provider calls.
P1	High-impact bug in an important workflow or surface. Serious for a role, route, grade band, provider, or release slice, but not a whole-platform emergency.	Fix before shipping the affected scope. Add targeted regression evidence and route to the owning session.	Teacher assignment page 404; one Visualization Lab template blank; AI tutor live mode fails without fallback; valid short answers rejected for one topic.
P2	Limited, recoverable, or polish defect. Quality matters, but core learning, data, release, and security are not currently at risk.	Queue or batch unless it sits in the active release path. Fix with focused evidence when touched.	Copy typo; minor mobile spacing issue; missing non-critical aria label; deterministic solver cannot prove an answer but no mismatch is known.

The Four Classification Questions

A good severity decision is a product decision, not just an engineering label. Start every triage with these four questions:

- **User harm:** Can a student, teacher, parent, or admin lose work, see unsafe output, be blocked from a promised workflow, or learn the wrong answer?
- **Blast radius:** Is it one browser, one route, one question, one curriculum track, one role, or the whole platform?
- **Release risk:** Would S22 release engineering or S11 QA be unable to approve the current release slice while this remains open?
- **Recoverability:** Can we repair it cleanly after the fact, or could data, credentials, costs, trust, or learning outcomes be damaged irreversibly?

Severity can move: Upgrade immediately when new evidence shows broader harm. Downgrade only after reproduction, logs, tests, or manual review prove the blast radius is narrow and recoverable.

P0 Bugs - Critical

Definition: A P0 is a stop-the-line bug. It means production use or release safety is not trustworthy because of outage, data loss, credential exposure, grading/content correctness at scale, unsafe student output, or a release-blocking build/deploy failure.

P0 triggers

- All or most users cannot register, log in, load the dashboard, or reach the core student learning flow.
- Real credentials, secrets, private corpus text, student data, or provider tokens appear in browser code, logs, reports, screenshots, or committed files.
- The active release build, type gate, runtime preflight, or production deployment path is red for the release candidate.
- Live practice/adaptive grading marks correct answers wrong, accepts wrong answers, or ships an answer-key mismatch across many students or a promoted content package.
- A live LLM, OCR, voice, email, or storage provider can run unbounded, leak data, bypass rate limits, or fail in a student-visible unsafe way.

Good P0 examples

Example	Why it fits this severity	Owner route and minimum proof

Auth/storage outage in production	Registration succeeds but later /api/me, login, or dashboard reads fail across Vercel instances. Identity and progress are unreliable.	Route to S12 plus S19; S22 holds release; S11 reruns authenticated smoke. Proof: durable storage health, repro gone, build/type gates green.
Build or deploy path cannot produce a safe release	An active release cannot build because of missing Next chunks, route imports, server/client boundaries, or generated-content imports.	Route to S22 plus S10/config and relevant feature owners. Proof: clean release slice builds and the blocker is recorded.
Live answer key mismatch or ambiguous MC item	A promoted question conflicts with the independent solution, or two options are defensibly correct. Existing content QA treats this as P0.	Route to S18/S04/S23; S11 covers practice regression. Proof: independent solution matches, exactly one option is correct, full QA reruns.
Credential or secret exposure	A DeepSeek, Qwen, Resend, database, Vercel, or library credential appears in client code, DOCX/report, screenshot, chat text, or Git history.	Route to S19 plus affected API owner. Proof: value redacted everywhere visible, rotated if credible, and reports keep only variable names.
Unbounded live provider or unsafe output	AI Tutor, adaptive rerank, OCR, voice, or email calls live providers without limits, fallback, or safety guardrails.	Route to S07/S15/S12 plus S19. Proof: rate limits, fallback, safe error state, and owner-approved live smoke when needed.

P1 Bugs - High

Definition: A P1 is a serious user-facing or release-scope bug in an important workflow. It blocks a role, route, curriculum band, or critical feature slice, but it does not currently show global outage, secret exposure, irreversible data loss, or platform-wide grading failure.

P1 triggers

- A common student, teacher, or parent workflow is blocked for a meaningful subset of users.
- An API route returns 404, 401, 500, stale data, or invalid schema for a shipped UI control, while the rest of the platform can continue.
- A key interactive learning surface is blank, broken, or mathematically wrong for one lab, game, or lesson module.
- AI, adaptive, analytics, or provider fallback behavior is broken for the affected surface but does not expose secrets or run unbounded.
- A red S11 E2E package blocks the affected feature release but is isolated to one owner package.

Good P1 examples

Example	Why it fits this severity	Owner route and minimum proof
Teacher workflow opens a missing page or endpoint	A shipped button routes to /teacher/assignments/new or a report endpoint that returns not-found/404. Student learning still works.	Route to S13 plus S12; S11 owns teacher regression. Proof: button reaches real page or is intentionally disabled; API schema/status passes.
One Practice Arena answer type rejects valid equivalents	Short-answer normalization rejects valid equivalent forms for one topic or grade band, while other question types still work.	Route to S04; S15 joins if adaptive locks depend on it; S18 if answer is disputed. Proof: focused alias test/manual repro passes.
AI Tutor live mode fails without useful fallback	DeepSeek text mode is configured, but the tutor throws or spins instead of showing local-helper mode or safe error.	Route to S07 plus S19. Proof: status route reports mode correctly and targeted tutor smoke passes without printing secrets.
Visualization Lab template is blank or misleading	One coordinate, graph, geometry, probability, or 3D lab loads but canvas is blank, controls overlap, or displayed values are wrong.	Route to S06; S18 confirms curriculum fit; S11/S22 verify gates. Proof: browser or Playwright visual/value check passes.
Adaptive next-step semantics are stale	Dashboard and API load, but recommendations ignore grade, curriculum profile, mastery, or recent attempts for a meaningful segment.	Route to S15 plus S02/S08 as needed. Proof: adaptive unit tests and targeted UI/API check reflect current learner state.

P2 Bugs - Medium or Low

Definition: A P2 is a limited, recoverable, or polish-level defect. It matters to product quality and should be tracked, but the core learning workflow, release path, student data, secrets, and grading correctness are not currently at serious risk.

P2 triggers

- The problem affects a rare viewport, old copy, a non-critical label, or an optional control while the primary action still works.
- A deterministic checker cannot prove a content answer, but there is no independent evidence of a wrong or ambiguous answer.
- A report, session log, QA artifact, or internal checklist needs cleanup but does not block a release decision.
- A minor accessibility, localization, visual rhythm, or sorting issue reduces polish but does not block the task.
- A test is flaky or under-specified but the product path still has reliable higher-level coverage.

Good P2 examples

Example	Why it fits this severity	Owner route and minimum proof
Copy, localization, or label inconsistency	A label says Practice in one place and Practice Arena elsewhere, or terminology is inconsistent while users complete the flow.	Route to S09 with the feature owner. Proof: copy reviewed in needed languages and no behavior change introduced.
Minor responsive spacing or polish issue	A card has extra whitespace, a badge wraps awkwardly, or a secondary panel needs spacing; primary controls remain usable.	Route to owning UI session, plus S09 for a11y/copy. Proof: manual or visual check confirms no clipping/overlap.
Content solver-gap needing manual review	The deterministic checker cannot prove an answer, but no mismatch, ambiguous MC, or content error has been found.	Route to S18; S21/S23 for candidate packages. Proof: manual proof or solver extension recorded; QA reruns with no P0/P1 row.
Non-blocking dashboard display drift	A progress card animates from stale cached state after reload, but saved progress, APIs, and recommendations are correct.	Route to S02; S08 joins if analytics semantics are wrong. Proof: focused UI check and unchanged underlying data.
Internal documentation cleanup	A session log misses a check line, a report has inconsistent wording, or this guideline needs a clearer example.	Route to S10. Proof: updated artifact is internally consistent; no code checks for documentation-only changes.

Escalation and Downgrade Rules

Escalate quickly when evidence touches these areas

- Secrets, real credentials, private datasets, student data, or account/session integrity.
- Production release, Vercel deployment safety, build/type gates, or S22 clean-slice readiness.
- Student learning correctness: answer keys, grading, adaptive recommendations, unsafe tutor output, or curriculum alignment.
- Multiple roles or surfaces failing together, such as login plus dashboard plus practice.
- Irreversible or hard-to-repair effects: data corruption, lost attempts, lost progress, or unbounded provider cost.

Downgrade only with evidence

- The defect is reproducible only in a narrow surface and does not affect active release scope.
- A reliable workaround exists and no data, secrets, cost, or grading correctness is at risk.
- Tests, logs, or manual QA show the affected population is small and recoverable.
- The owner has accepted a temporary disable/hide decision for a non-core feature.

Pattern rule: Several P2 bugs in the same user journey can become one P1. Several P1 bugs across the core student journey can become a P0 release blocker, even when each individual defect looks isolated.

Debugging Workflow for New Employees

1. **Reproduce narrowly first.** Use the smallest route, account, grade, topic, viewport, provider mode, or API call that shows the failure. Record exact inputs and expected vs actual behavior.
2. **Classify tentatively.** Pick P0/P1/P2 using the four questions. State what evidence would change the severity.
3. **Route to the owning session.** Use AGENTS.md. Examples: S04 for Practice Arena, S06 for Visualization Lab, S07 for AI Tutor, S12 for backend/API platform, S18 for content QA, S22 for release engineering, S25 for non-destructive Git hygiene.
4. **Protect the worktree and secrets.** Run git status before editing, do not revert unrelated changes, and never print or copy real credential values.
5. **Fix the smallest cause.** Prefer a direct, testable repair in the owning scope before broad refactors. For content, do not bypass S18/S23 promotion gates.
6. **Verify at the right level.** P0 usually needs repro proof plus type/build/release or content QA evidence. P1 needs targeted regression. P2 needs a focused check or documented manual review.
7. **Write the handoff.** List changed files, checks run, checks not run, assumptions, risks, and follow-up owner/session. Severity labels without evidence are not enough.

Owner Routing Cheat Sheet

Bug area	Primary owner	Typical severity clues
App shell/auth entry	S01 UI shell; S12 auth/API when server route; S19 env if secret/config	P0 if login/register broadly fails; P1 if one shell route or selector fails.
Dashboard/ progress	S02 UI; S08 analytics/shared state; S15 adaptive semantics	P1 if progress/adaptive display blocks core guidance; P2 for minor stale animation.
Practice/question data	S04 practice; S18 content QA; S23 promotion when candidate-to-live	P0 for live wrong/ambiguous answer at scale; P1 for one workflow/answer-type regression.
Lessons/textbooks	S05 lessons; S18/S21/S23 for content packages	P0/P1 when content correctness or release package safety is compromised.
Visualization Lab	S06 visualization; S18 for curriculum fit; S11/S22 for gates	P1 if a lab is blank or math values are wrong; P2 for minor spacing.

AI/adaptive APIs	S07 AI Tutor; S15 adaptive; S12 backend routes; S19 env	P0 for secret leak/unbounded provider; P1 for broken fallback or one provider mode.
Teacher/parent consoles	S13 teacher; S14 parent; S12 backend routes	P1 for shipped buttons to missing routes/endpoints; P2 for copy or low-risk display.
Release/build/e2e	S22 release; S11 QA; S10 config/docs; S25 Git hygiene	P0 for active release build/deploy block; P1 for isolated regression package.
Docs/reports	S10 docs/report lead	Usually P2 unless incorrect report data drives a release or owner decision.

Bug Report Template

Use this template in issue comments, session logs, blocker reports, or handoffs. Keep secrets redacted and use exact file/route names when possible.

Title	Short user-impact statement, not implementation guess.
Severity	P0/P1/P2 plus one sentence explaining user harm, blast radius, release risk, and recoverability.
Owner route	Primary Sxx owner and any coordinating sessions.
Affected surface	Route, API, component, data package, role, grade, provider mode, or viewport.
Steps to reproduce	Minimal exact steps. Include account type and environment, but never credentials.
Expected result	What should happen for the learner, teacher, parent, owner, or release gate.
Actual result	What happens now, with redacted logs or screenshots if needed.
Evidence	Test failure, screenshot path, report row, log excerpt, or manual check. Redact secrets.
Proposed verification	Targeted unit/API/UI/e2e/build/content QA check required before closing.
Stop conditions	Owner decision, secret rotation, content signoff, release hold, or scope conflict if relevant.

Common Mistakes to Avoid

- Do not downgrade a P0 because the fix looks easy. Severity follows impact, not effort.
- Do not call a bug P0 just because it is annoying. Prove release, data, secret, cost, correctness, or broad user impact.
- Do not paste secrets, API responses with tokens, private corpus text, or real credentials into reports or screenshots.

- Do not fix outside your assigned write scope. Write a blocker or route to the owning session instead.
- Do not bypass S18/S23 content gates by editing live question or lesson data directly from a candidate package.
- Do not treat a red broad E2E run as one giant bug. S11 should split failures into owner-routed regression packages.

Closing Rule

Default posture: Protect learners first, protect secrets always, protect the release path, and then make the smallest evidence-backed fix. A clear P2 report is better than a vague P0 alarm; a fast P0 escalation is better than a quiet production incident.